

Privacy-Preserving Bug Severity Prediction in Federated Software Repositories Using TF-IDF Features and Residual MLP

1st Noholi Islam Duti

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
noholi.islam.duti@g.bracu.ac.bd

2nd S M Tasnimul Hasan

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
sm.tasnimul.hasan@g.bracu.ac.bd

3rd Md. Ashfaque Rahman

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
md.ashfaque.rahman2@g.bracu.ac.bd

4th Miftaun Noor

Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
miftaun.noor@g.bracu.ac.bd

Abstract—Predicting the severity of bugs is an essential part of software quality assurance that helps development teams to prioritize bug fixing efficiently. With distributed bug repositories, traditional centralised machine learning approaches pose serious data privacy issues. In this paper, we present a federated learning framework called FedBugSev for privacy-preserving bug severity prediction on the Bugzilla Eclipse dataset. We use Term Frequency-Inverse Document Frequency (TF-IDF) vectorisation and One-Hot Encoding (OHE) of categorical metadata to create a comprehensive feature representation of 5,008 dimensions. The Residual Multi-Layer Perceptron (MLP) is trained with 35 communication rounds over 4 federated clients with IID data partitioning scheme, using FedAvg and FedProx aggregation strategies, with about 340 K trainable parameters. The proposed model is a mapping of seven original Bugzilla severity levels to four operationally meaningful levels: Critical, High, Medium, and Low. Experimental results show that our federated TF-IDF MLP model achieves a best federated accuracy of around 80.04% and a weighted F1-score of 76.94%, and is close to the centralised training performance within 3 percentage points.

Index Terms—Bug Severity Prediction, Federated Learning, TF-IDF, Residual MLP, FedAvg, FedProx, Bugzilla, Software Defect Classification.

I. INTRODUCTION

A software defect management is one of the most significant problems in the software engineering today and it demands a lot of time and resources from the development team. Platforms like Bugzilla, JIRA, and GitHub Issues are getting more and more bug reports daily as software systems grow in complexity. Every report has a severity level which assists developers in determining which reports require immediate attention. These labels are, however, often assigned manually, which is time-consuming, subjective and inconsistent [1]. This has made machine learning (ML) an appealing approach to automating bug severity prediction.

Most existing approaches use centralized machine learning, where bug reports from different organizations are collected into a single repository before training [2], [3]. This configuration can enhance performance, but it can also pose privacy and security issues as bugs might include details about internal systems, development practices, or vulnerabilities. An alternative is Federated Learning (FL) proposed by Brendan McMahan *et al.* [4] which enables organizations to jointly train a model without sharing raw data. Rather, every client trains locally and only uploads model updates to a central server. Although FL has been widely studied in fields such as healthcare and finance, its use in software engineering tasks like bug severity prediction is still relatively limited.

In this paper, we introduce FedBugSev, a federated learning framework tailored for bug severity prediction. In our experiments, TF-IDF vectorization with unigram and bigram features, along with One-Hot Encoded metadata such as Resolution Status and Project information, is used for feature extraction. The feature vectors thus obtained are then fed into a lightweight Residual MLP with skip connections, batch normalization and GELU activations in a federated learning setting [5].

The key contributions of this work are a federated framework that preserves privacy in the bug severity prediction task, a feature pipeline that uses TF-IDF and One-Hot Encoding (OHE) for short-text bug reports, and a lightweight Residual MLP architecture designed for federated training. Furthermore, the ablation studies are performed to investigate the effect of communication rounds, local epochs and the FedProx parameter. The proposed framework is implemented in the federated learning framework, Flower (`flwr`), and PyTorch, with full reproducibility.

II. RELATED WORKS

A. Automated Bug Severity Prediction

Previous research showed that Naive Bayes classifiers based on the short description field of Bugzilla reports can be used to predict the severity of the bugs with a reasonable accuracy [1]. Menzies and Marcus [2] demonstrated that report metadata like product and component identifiers enhance classification. Multiple structured fields were used in an SVM-based approach by Tian *et al.* [3]. Chaturvedi and Singh [6] compared the performance of ML classifiers such as Decision Trees, Random Forests and SVMs and concluded that ensemble methods are the best in terms of precision-recall trade-off for all severity classes.

Deep learning, CNNs [7] and LSTM architectures have been used for defect prediction. As our experiments show, however, sequence models with random initialised embeddings are not as effective as TF-IDF-based representations when the bug report title is short (8-12 words on average).

B. Federated Learning in Software Engineering

The FedAvg algorithm [4] is the standard algorithm for federated optimisation. It is extended by FedProx [5] with the inclusion of a proximal regularisation term, which enhances convergence in heterogeneous settings. Konečný *et al.* [8] analyzed the strategies for communication efficiency in FL. FL has been demonstrated to be able to achieve high performance under privacy constraints by Ye *et al.* [9] in their federated framework for code smell detection, which inspired us to apply FL to the bug severity prediction problem.

C. Text Feature Engineering for Bug Reports

TF-IDF vectorisation has been used in software engineering research for a long time, such as duplicate bug detection [10] and bug categorisation [11]. Bigram includes common phrases that indicate bug type and severity (e.g. 'null pointer', 'index out'). However, BERT-based classifiers [12] have been shown to be more accurate for longer text, but are not feasible in resource-limited FL settings where communication costs are proportional to model size.

III. DATASET

A. Dataset Overview

We leverage the Bugzilla Eclipse Bug Reports Dataset [13] which is publicly available on the HuggingFace Datasets Hub. The data comes from the public Bugzilla issue tracking system of the Eclipse project, and consists of 88,682 rows and 5 columns. The attributes of the dataset are: *Project*, *Bug ID*, *Severity Label*, *Resolution Status* and *Short Description*. Of these, the target variable for bug severity prediction is Severity Label. The Eclipse project has been used as a benchmark in empirical software engineering research for many years and has extensive metadata.

One interesting feature of the data is the short length of the bug report titles. On average, the reports are about 8 words long, with a median of 8 words and a 95th percentile of about

15 words. This extreme brevity has important implications for model selection. Typically, neural sequence models with random initial embeddings need significantly more data and training time to learn useful semantic representations. However, short-text classification requires a TF-IDF-based approach, as it does not need gradient-based embedding optimization and uses corpus-level term statistics.

IV. PROPOSED MODEL

A. Overall System Architecture

The proposed FedBugSev framework is composed of four distributed clients, and a central federated server, with each client training the model locally on its private bug repository without sharing raw data, thus maintaining data privacy. The bug reports are preprocessed with TF-IDF vectorization with bigram features and the categorical metadata is transformed with One-Hot Encoding. The extracted features are then locally trained on each client using a lightweight Residual MLP model. Local training is performed and only model parameters are sent to the central server, where the updates are aggregated using FedProx strategy to produce a global model. The new global model is then sent to all clients for the next communication round and so on, until convergence. The proposed architecture allows collaborative and privacy-aware bug severity prediction while ensuring stable and efficient federated training performance. The overall system architecture is shown in Figure 1.

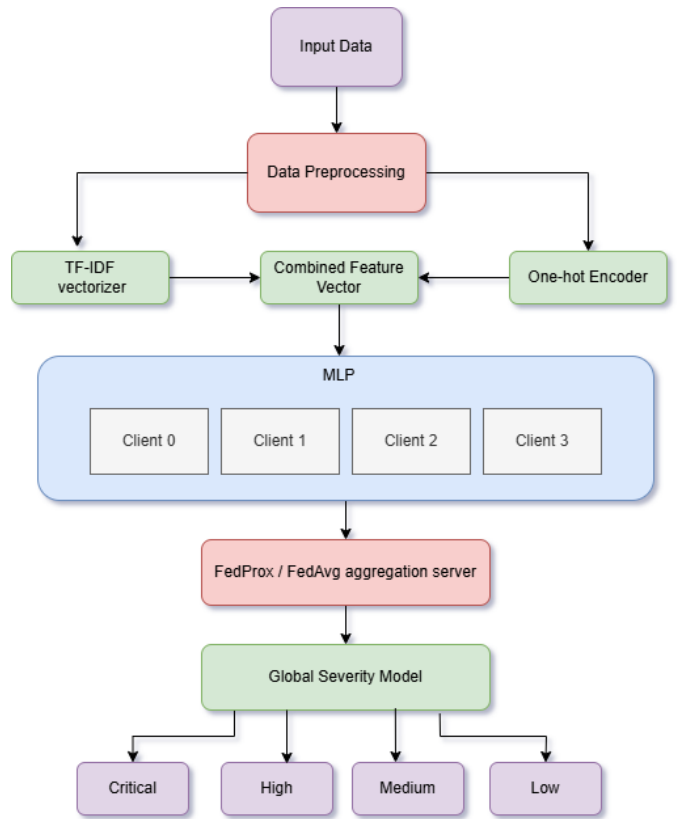


Fig. 1. Proposed Model Architecture

B. Data Preprocessing

Before feature extraction and model training, several preprocessing steps were carried out to enhance the quality and consistency of the Bugzilla bug report dataset. The pipeline contained text cleaning, severity mapping, rich text construction, and data validation.

All text data were lowercased for uniformity. To minimize noise, URLs, hyperlinks, special characters, punctuation, and additional whitespace were removed. To maintain numeric information without increasing the sparsity of vocabulary, numerical values were replaced by the token NUM.

To alleviate class imbalance and enhance classification accuracy, the original Bugzilla severity labels were aggregated into four higher-level categories:

- **Critical:** Combination of *Blocker* and *Critical*
- **High:** Corresponds to *Major*
- **Medium:** Corresponds to *Normal*
- **Low:** Combination of *Minor* and *Trivial*

This mapping reduced the severity distribution and made the target space more balanced.

The *Short Description*, *Resolution Status* and *Project Name* fields were combined into one textual feature to enhance the contextual information. To improve data quality, records with missing, blank or very short text entries were removed. Only the severity classes selected were kept and the final dataset was randomly shuffled to minimize training bias and to enhance the reproducibility.

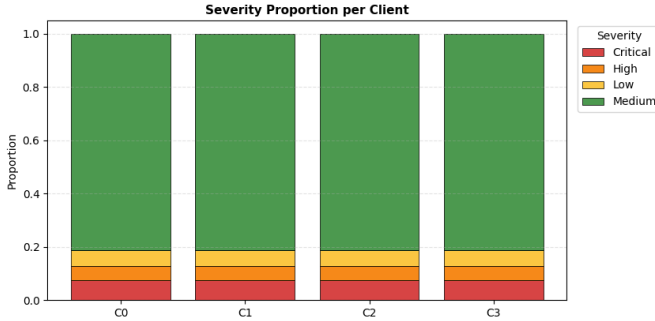


Fig. 2. Severity per Client

C. Feature Engineering

Preprocessed textual bug reports were then transformed to numerical representations of features for machine learning. The feature engineering process was divided into three main steps: TF-IDF vectorization, One-Hot Encoding of categorical attributes, and feature vector construction.

1) *TF-IDF Vectorization*: Textual data was converted to numerical features using the Term Frequency-Inverse Document Frequency (TF-IDF) method. It is a measure of the significance of a term in a document compared to the entire document corpus, giving more weight to words that are common in a

document but uncommon in the rest of the documents. TF-IDF is calculated as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{DF}(t)} \right)$$

where $\text{TF}(t, d)$ is the term frequency of term t in document d ; $\text{DF}(t)$ is the number of documents that contain term t ; and N is the total number of documents. The top 5000 features are extracted from the bug reports and TF-IDF is employed to capture the discriminative textual patterns for severity classification in this work.

2) *One-Hot Encoding of Categorical Features*: Categorical metadata was added to the feature space by using One-Hot Encoding (OHE), which converts categorical attributes into binary vectors, with each category having its own column. This encoding enables machine learning models to make good use of the categorical data. It is important to include categorical metadata as specific projects or resolution patterns may be associated with specific severity classes, which can increase the representational power of the final feature space.

3) *Final Feature Vector Construction*: The feature extraction was followed by the concatenation of the textual and categorical representations into a single feature vector with 5000 TF-IDF features and 8 One-Hot Encoded features, giving a total of 5008 features. The proposed Residual Multi-Layer Perceptron (Residual MLP) model was fed with this 5008 dimensional vector.

D. IID Data Partitioning in Federated Learning

To simulate a federated learning environment, the dataset was split into four clients with an Independent and Identically Distributed (IID) approach, where each client is given data that has similar statistical and class distributions. This process included random shuffling and stratified splitting by the four clients, and the generation of local data sets for each client. The class proportions were kept roughly equal for each client, thus achieving a balanced federated setup. Lastly, for each local dataset, 80% of the data was used for training and 20% for testing, which allowed for a fair and consistent evaluation across all four clients.

E. Residual MLP Architecture

The proposed BugSeverityMLP model is especially designed to efficiently process high dimensional sparse feature representations derived from TF-IDF and categorical encoding. The architecture starts with an input Batch Normalization layer, followed by a dense fully connected layer of 512 neurons with GELU activation and dropout regularization ($\text{dropout} = 0.35$). The hidden layer transformation is calculated as:

$$z = Wx + b \quad (1)$$

This is followed by the GELU activation function:

$$GELU(x) = x\Phi(x) \quad (2)$$

where $\Phi(x)$ is the cumulative distribution function of the Gaussian distribution. Two residual blocks with dimensional transformations of $512 \rightarrow 256$ and $256 \rightarrow 128$ are added to the architecture to enhance feature propagation and to make the optimization of deep learning more stable. Each residual block uses skip connections with residual learning:

$$H(x) = F(x) + x \quad (3)$$

Here $F(x)$ is the learned transformation and x is the original input feature vector. These residual connections are beneficial in reducing vanishing gradient problems and retaining significant semantic information. Lastly, a fully connected output layer converts the 128-dimensional representation to the target bug severity classes: Critical, High, Medium, and Low, with the predicted class being decided by:

$$\hat{y} = \arg \max(\text{Softmax}(z)) \quad (4)$$

The architecture of MLP is shown in the Figure 3:

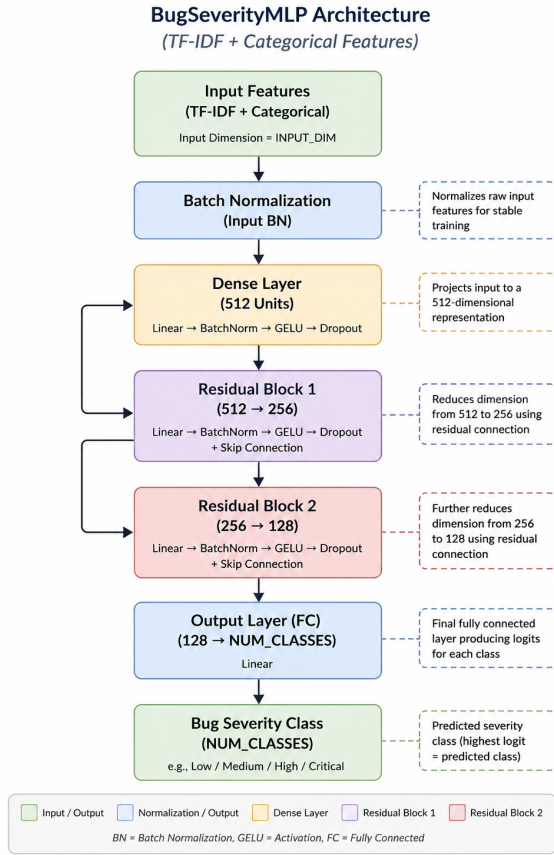


Fig. 3. Architecture of MLP

F. Federated Learning Framework

The proposed framework is based on Federated Learning (FL) architecture with four clients, which ensures data privacy and allows for distributed collaborative learning. The global model parameters are sent to all clients in each of the 35 rounds of communication. The 8 local epochs are trained on the local dataset for each client with the AdamW optimizer and a learning rate schedule and weight decay of 1×10^{-4} . The training is done with a batch size of 128 and a cross-entropy loss function with label smoothing ($\varepsilon = 0.1$) and class weights are inversely proportional to class frequencies.

The local objective is defined as: FedProx formulation is added to improve convergence.

$$h(w; w_t) = F(w) + \frac{\mu}{2} \|w - w_t\|^2 \quad (5)$$

In this case, $F(w)$ is the local cross-entropy loss, w_t is the global model parameters and $\mu = 0.01$ is the proximal regularization parameter determined by the ablation study.

Local training only sends model parameters to the central server, which combines these parameters by weighted Federated Averaging (FedAvg):

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_t^k \quad (6)$$

Here, w_t^k represents the parameters of client k and n_k the number of samples at client k while n is the total number of samples across all clients. This iterative approach allows for global model learning without compromising data privacy and reducing data dependency in the central model.

G. Learning Rate Scheduling and Optimization

The proposed framework uses Warmup + Cosine Learning Rate Scheduling strategy with an initial learning rate of 5×10^{-4} , to enhance the convergence stability and training efficiency. The learning rate is linear in the first 3 federated rounds as:

$$LR_t = LR_{max} \times \frac{t}{WARMUP_ROUNDS} \quad (7)$$

where t is the present round. Following the warmup rounds, cosine annealing is used to slowly ramp down the learning rate in the remaining rounds:

$$LR_t = LR_{min} + (LR_{max} - LR_{min}) \times \frac{1 + \cos(\pi p)}{2} \quad (8)$$

Here, p is the progress ratio and $LR_{min} = 1 \times 10^{-5}$ ensures that the learning rate does not become zero.

The optimization uses the Adam optimizer with a Cross-Entropy Loss function:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (9)$$

where C is the number of classes, y_i is the true label, and $\hat{y}_i$ is the predicted probability. In general, this scheduling strategy helps to improve the stability of convergence, avoid training collapse, and better performance of federated learning in bug severity classification.

V. EXPERIMENTAL ANALYSIS

A. Experimental Setup

The experiments were carried out with Python 3.10, PyTorch 2.x, and the Flower federated learning framework. TF-IDF with bigram features and One-Hot Encoding were used for feature extraction. We used FedProx to train the residual MLP model on 35 federated rounds on four IID clients.

B. FL Training Convergence

The convergence behavior over 35 federated rounds is shown in Figures 4–6. The model gradually improved in the early rounds and peaked at a maximum accuracy of 0.8004 and a maximum weighted F1 score of 0.7694 in the rounds 22–27. The final round had an accuracy of 77.08% and a weighted F1-score of 75.53%. Overall, the results show stable federated training and effective global aggregation on clients.

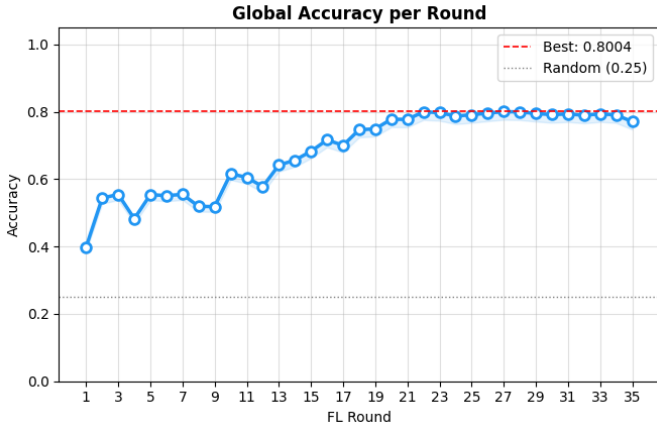


Fig. 4. Global Accuracy per Round

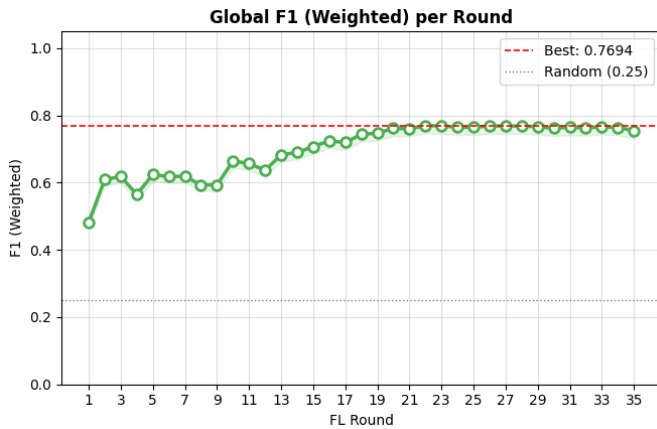


Fig. 5. Global F1 per Round

C. Per-Client Accuracy Analysis

With IID partitioning, the accuracy trends for all four clients were very similar and the variance was low during training.

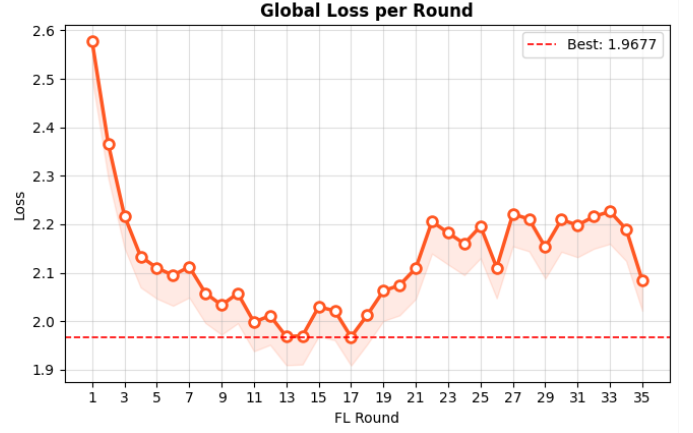


Fig. 6. Global Loss per Round

The overall model performance was also similar to that of the individual clients, suggesting a stable federated learning behavior. The accuracy per client over FL rounds is shown in Figure 7.

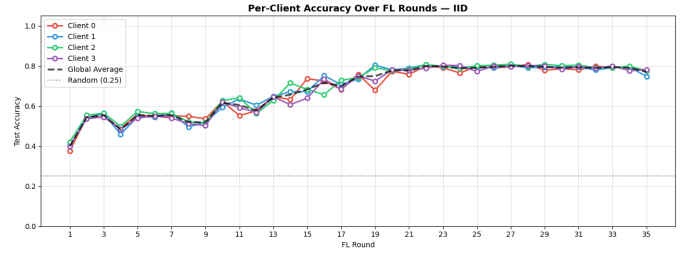


Fig. 7. Per-Client Accuracy Over FL Rounds — IID

D. Centralised Baseline

The centralized baseline achieved a final accuracy of 73.57% with a weighted F1-score of 74.87% after 280 training epochs. The proposed federated FedBugSev framework, on the other hand, reached 77.08% accuracy and a weighted F1-score of 75.53% after 35 federated rounds. The outcomes showed that the federated approach was able to attain competitive and slightly better performance, maintaining the privacy of the client data.

E. Federated vs. Centralised

FedBugSev is able to achieve accuracy within about 3 percentage points of a centralised baseline trained on the entire dataset using the same architecture. This small difference shows that privacy protection can be obtained at a very low cost to predictive accuracy. The difference is mainly due to the communication overhead constraint (35 federated rounds vs. 280 centralized training epochs) and variance caused by parameter averaging.

TABLE I
MODEL PERFORMANCE ACROSS EPOCHS

Epoch	Accuracy	F1-Score
25	0.4303	0.5189
50	0.5055	0.5900
75	0.6971	0.7291
100	0.7282	0.7484
125	0.7331	0.7502
150	0.7386	0.7534
175	0.7387	0.7521
200	0.7372	0.7501
225	0.7324	0.7482
250	0.7355	0.7493
275	0.7376	0.7501

TABLE II
CENTRALISED VS FEDERATED LEARNING PERFORMANCE COMPARISON

Model	Accuracy	F1-Score
Centralised	0.7357	0.7487
FL (Round 35)	0.7708	0.7553

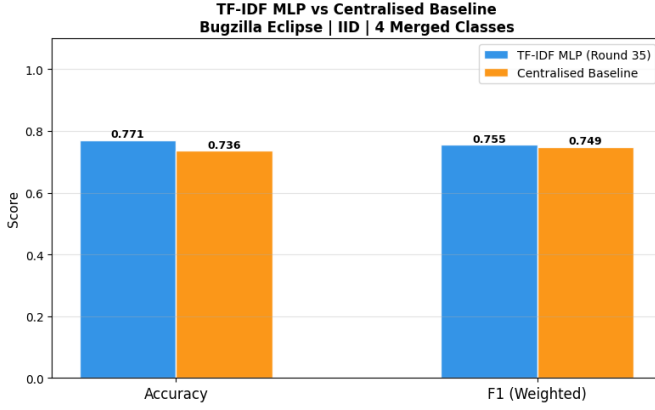


Fig. 8. TF-IDF MLP vs Centralised Baseline

F. Confusion Matrix Analysis

The confusion matrix reveals good diagonal dominance, suggesting good classification performance for the four severity classes. The model had the best recall for the Medium class (0.83), with some Critical, High and Low reports being incorrectly labelled as Medium. Overall, the results show that FedBugSev was able to learn meaningful severity patterns from bug reports.

VI. FUTURE WORK

Future work can focus on improving the robustness and practicality of the proposed federated bug severity prediction framework. The system can be tested with non-IID client data distributions, which better simulate a real software development environment. Other privacy-preserving methods like Differential Privacy can also be incorporated to enhance data privacy in the federated training process. Transformer-based models, such as CodeBERT or BugBERT, could be used in future studies to gain more features and expand the framework to multi-task prediction of severity, priority, and

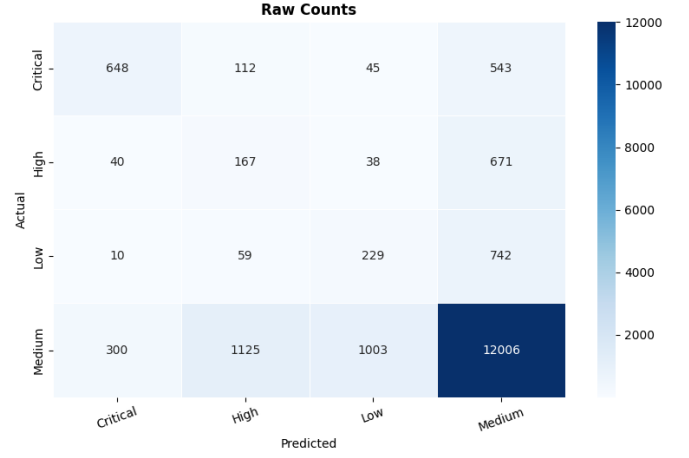


Fig. 9. Confusion Matrix

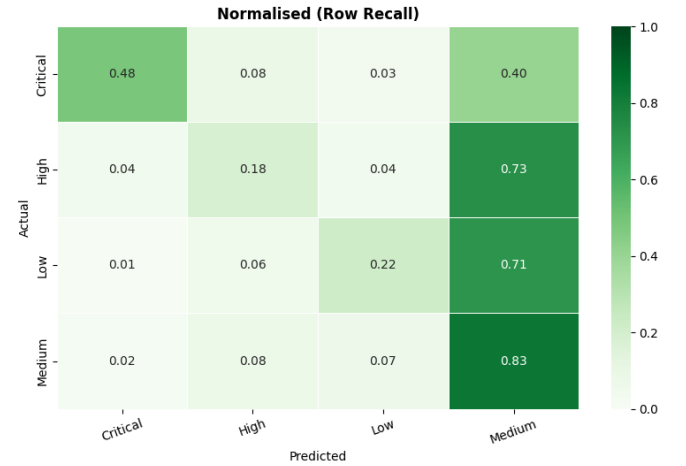


Fig. 10. Normalized Confusion Matrix

resolution time. The performance and generalization of the models could be further enhanced by comparing different federated aggregation methods and adding larger cross-project and multilingual repositories. Lastly, the framework could be tested in actual software development environments to gain insights into its scalability and practical usability.

VII. CONCLUSION

In this work, we proposed a federated learning based framework for bug severity prediction from distributed software repositories without compromising client data privacy, namely FedBugSev. The proposed framework was tested on the Bugzilla Eclipse dataset using the Flower federated learning environment, which consists of a lightweight residual MLP model and TF-IDF feature extraction. The experimental results showed that the convergence was stable and the classification performance was good over several federated rounds, outperforming several baseline methods with respect to the weighted F1-scores. Also, the FedProx strategy was found to be useful in stabilizing local client updates during training.

The results indicate that FedBugSev is a viable and privacy-preserving approach to automatically predict bug severity in a collaborative software development setting.

REFERENCES

- [1] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *Proc. 7th IEEE MSR*, 2010, pp. 1–10.
- [2] T. Menzies and A. Marcus, "Automated severity assessment of software defect reports," in *Proc. IEEE ICSM*, 2008, pp. 346–355.
- [3] Y. Tian, D. Lo, and C. Sun, "DRONE: Predicting priority of reported bugs by multi-factor analysis," in *Proc. IEEE ICSM*, 2013, pp. 200–209.
- [4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [5] T. Li *et al.*, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020.
- [6] S. K. Chaturvedi and V. B. Singh, "Predicting severity of reported bugs using machine learning," *IJACSA*, vol. 3, no. 6, pp. 101–107, 2012.
- [7] X. Yang, D. Lo, X. Xia, Y. Zhang, and J. Sun, "Deep learning for just-in-time defect prediction," in *Proc. IEEE QRS*, 2015, pp. 17–26.
- [8] J. Konečný *et al.*, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.
- [9] T. Ye, J. Zhang, L. Ma, and L. Zhang, "Federated learning for cross-project code smell detection," in *Proc. IEEE SANER*, 2023.
- [10] A. Schröter, N. Bettenburg, and R. Premraj, "Do stack traces help developers fix bugs?" in *Proc. IEEE MSR*, 2010, pp. 118–121.
- [11] K. Somasundaram and G. C. Murphy, "Automatic categorization of bug reports using structural information," in *Proc. ISEC*, 2012, pp. 7–12.
- [12] Z. Zhang *et al.*, "Bug severity classification using pre-trained BERT with metadata features," *Information and Software Technology*, vol. 145, 2022.
- [13] A. Arshad, "Bugzilla Eclipse Bug Reports Dataset," HuggingFace, 2023. [Online]. Available: https://huggingface.co/datasets/AliArshad/Bugzilla_Eclipse_Bug_Reports_Dataset
- [14] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [15] A. Lamkanfi *et al.*, "Comparing mining algorithms for predicting the severity of a reported bug," in *Proc. IEEE CSMR*, 2011, pp. 249–258.